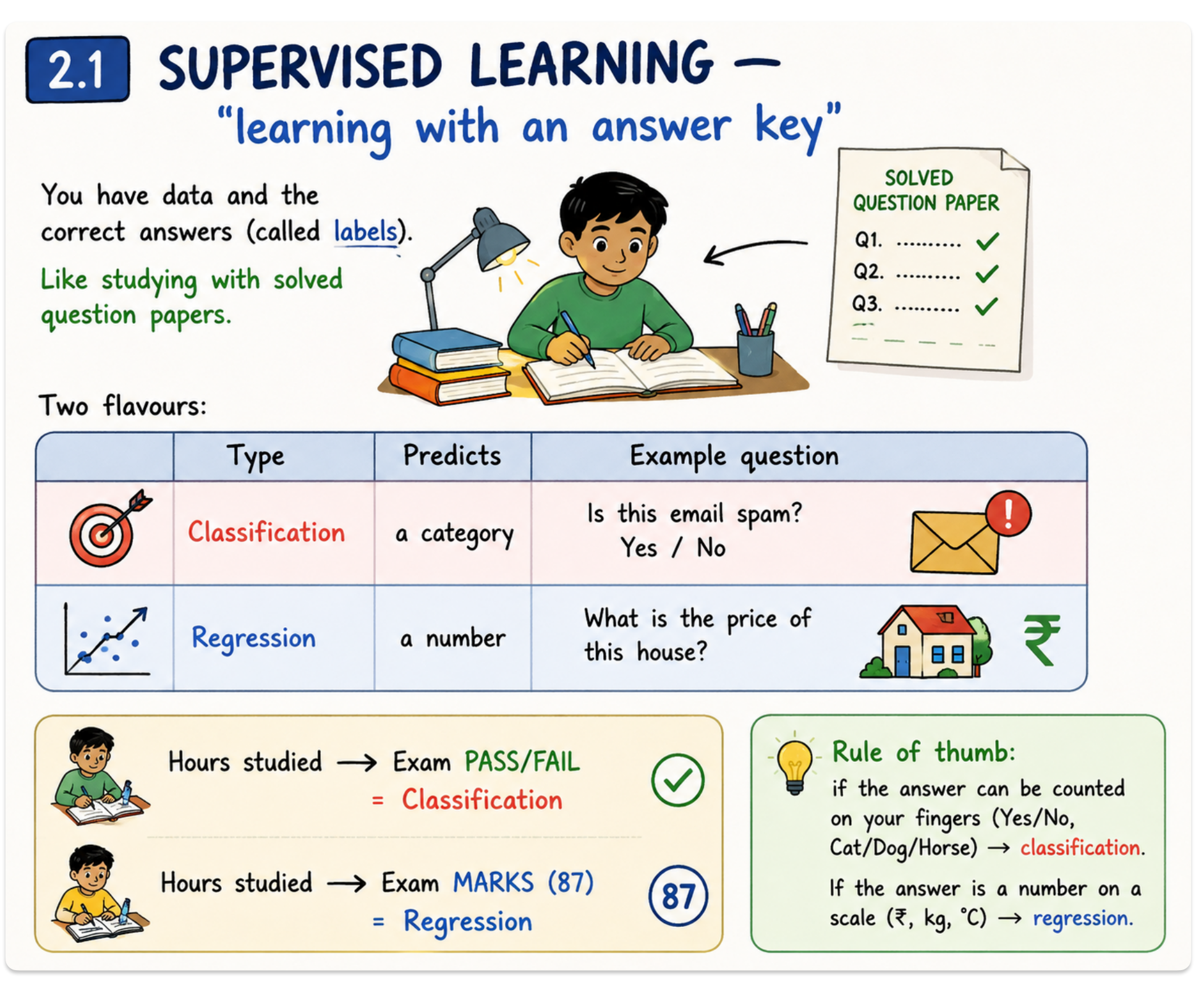
Machine Learning



2.1 SUPERVISED LEARNING — "learning with an answer key"

You have data and the correct answers (called labels).

Like studying with solved question papers.

SOLVED QUESTION PAPER
Q1. .... ✓
Q2. .... ✓
Q3. .... ✓

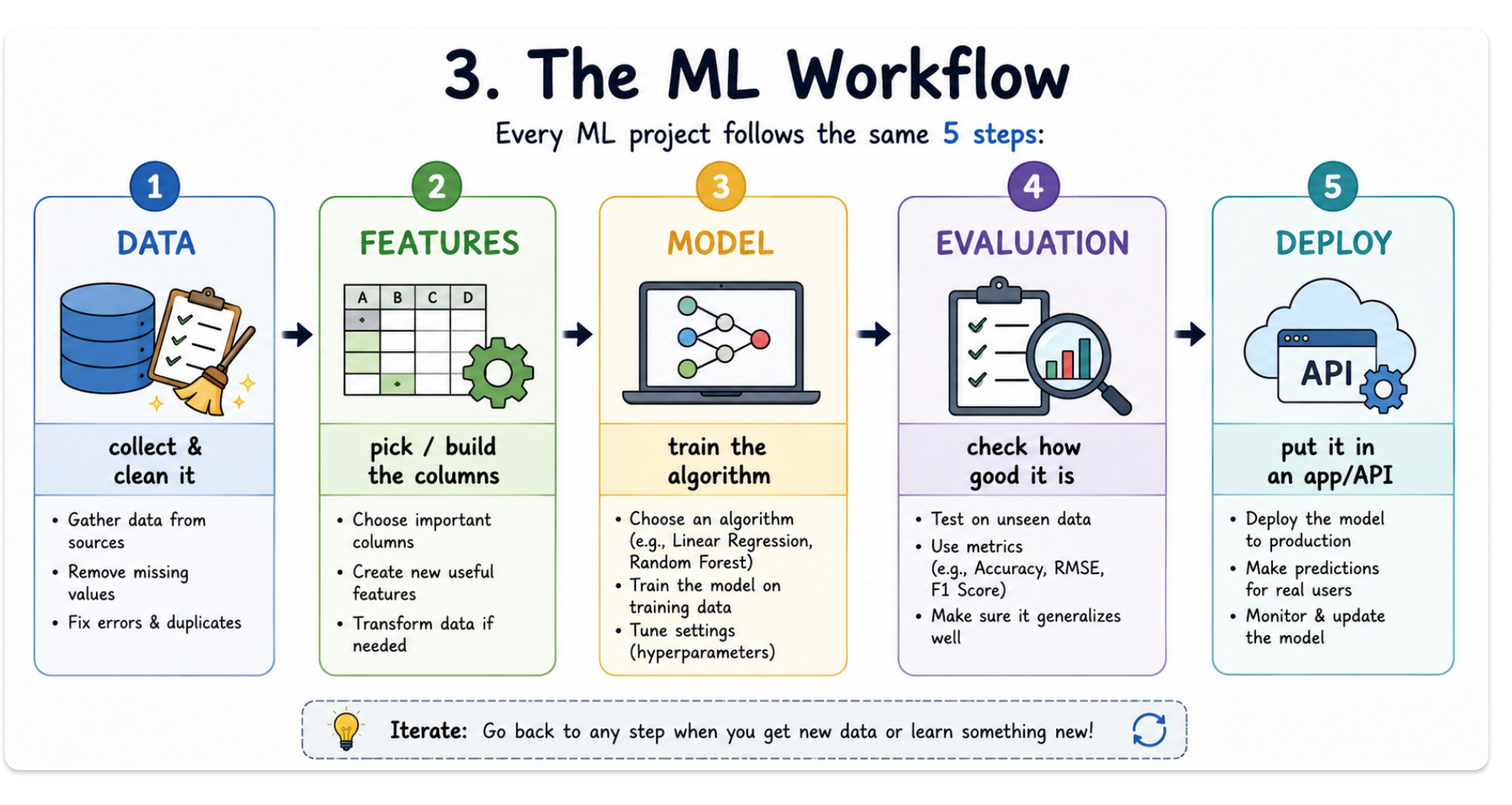
Two flavours:

	Type	Predicts	Example question
	Classification	a category	Is this email spam? Yes / No
	Regression	a number	What is the price of this house?

Hours studied → Exam PASS/FAIL = Classification ✓

Hours studied → Exam MARKS (87) = Regression

Rule of thumb: if the answer can be counted on your fingers (Yes/No, Cat/Dog/Horse) → classification. If the answer is a number on a scale (₹, kg, °C) → regression.

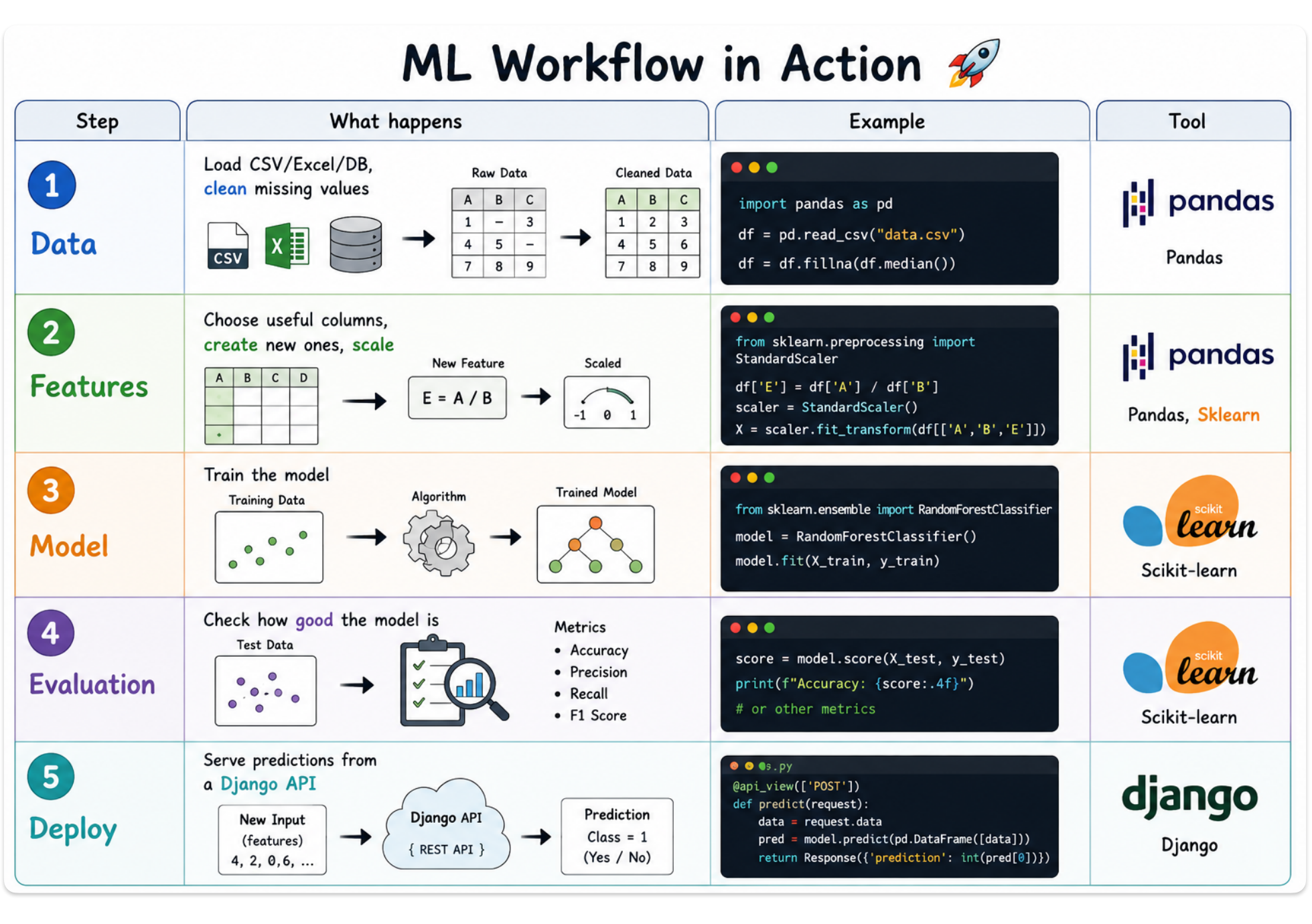


3. The ML Workflow

Every ML project follows the same 5 steps:

1. **DATA** — collect & clean it
   - Gather data from sources
   - Remove missing values
   - Fix errors & duplicates
2. **FEATURES** — pick / build the columns
   - Choose important columns
   - Create new useful features
   - Transform data if needed
3. **MODEL** — train the algorithm
   - Choose an algorithm (e.g., Linear Regression, Random Forest)
   - Train the model on training data
   - Tune settings (hyperparameters)
4. **EVALUATION** — check how good it is
   - Test on unseen data
   - Use metrics (e.g., Accuracy, RMSE, F1 Score)
   - Make sure it generalizes well
5. **DEPLOY** — put it in an app/API
   - Deploy the model to production
   - Make predictions for real users
   - Monitor & update the model

Iterate: Go back to any step when you get new data or learn something new!



ML Workflow in Action 🚀

1 Data — Load CSV/Excel/DB, clean missing values — Tool: Pandas

```
import pandas as pd
df = pd.read_csv("data.csv")
df = df.fillna(df.median())
```

2 Features — Choose useful columns, create new ones, scale — Tool: Pandas, Sklearn

```
from sklearn.preprocessing import StandardScaler
df['E'] = df['A'] / df['B']
scaler = StandardScaler()
X = scaler.fit_transform(df[['A','B','E']])
```

3 Model — Train the model — Tool: Scikit-learn

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

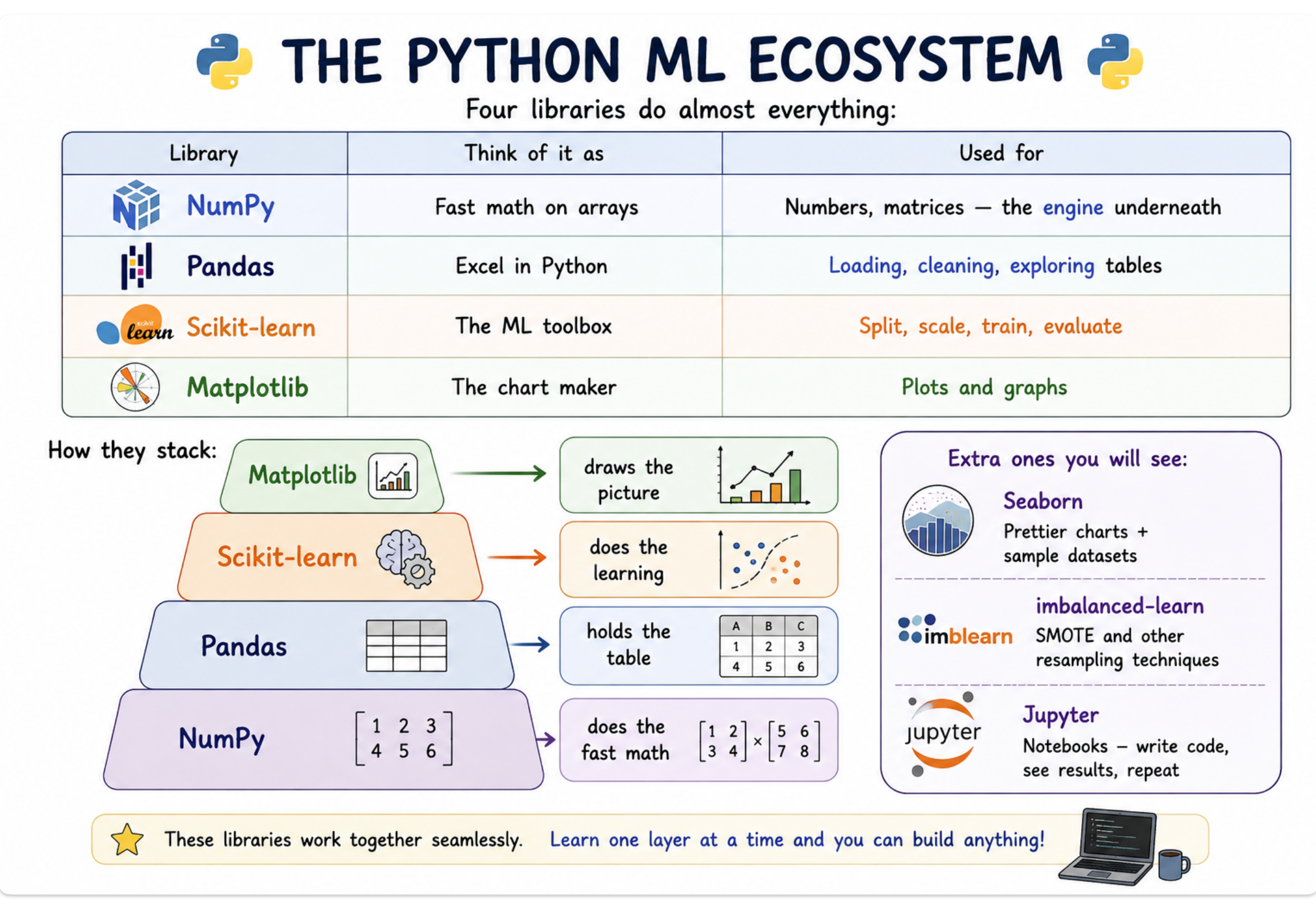
4 Evaluation — Check how good the model is — Metrics: Accuracy, Precision, Recall, F1 Score — Tool: Scikit-learn

```
score = model.score(X_test, y_test)
print(f"Accuracy: {score:.4f}")
# or other metrics
```

5 Deploy — Serve predictions from a Django API — Tool: Django

```
@api_view(['POST'])
def predict(request):
    data = request.data
    pred = model.predict(pd.DataFrame([data]))
    return Response({'prediction': int(pred[0])})
```

Step
What happens
Example
Tool
Raw Data
Cleaned Data
CSV
New Feature
E = A / B
Scaled
Training Data
Algorithm
Trained Model
Test Data
New Input (features) 4, 2, 0.6, ...
Django API { REST API }
Prediction Class = 1 (Yes / No)
pandas
learn
django



THE PYTHON ML ECOSYSTEM

Four libraries do almost everything:

Library	Think of it as	Used for
NumPy	Fast math on arrays	Numbers, matrices — the engine underneath
Pandas	Excel in Python	Loading, cleaning, exploring tables
Scikit-learn	The ML toolbox	Split, scale, train, evaluate
Matplotlib	The chart maker	Plots and graphs

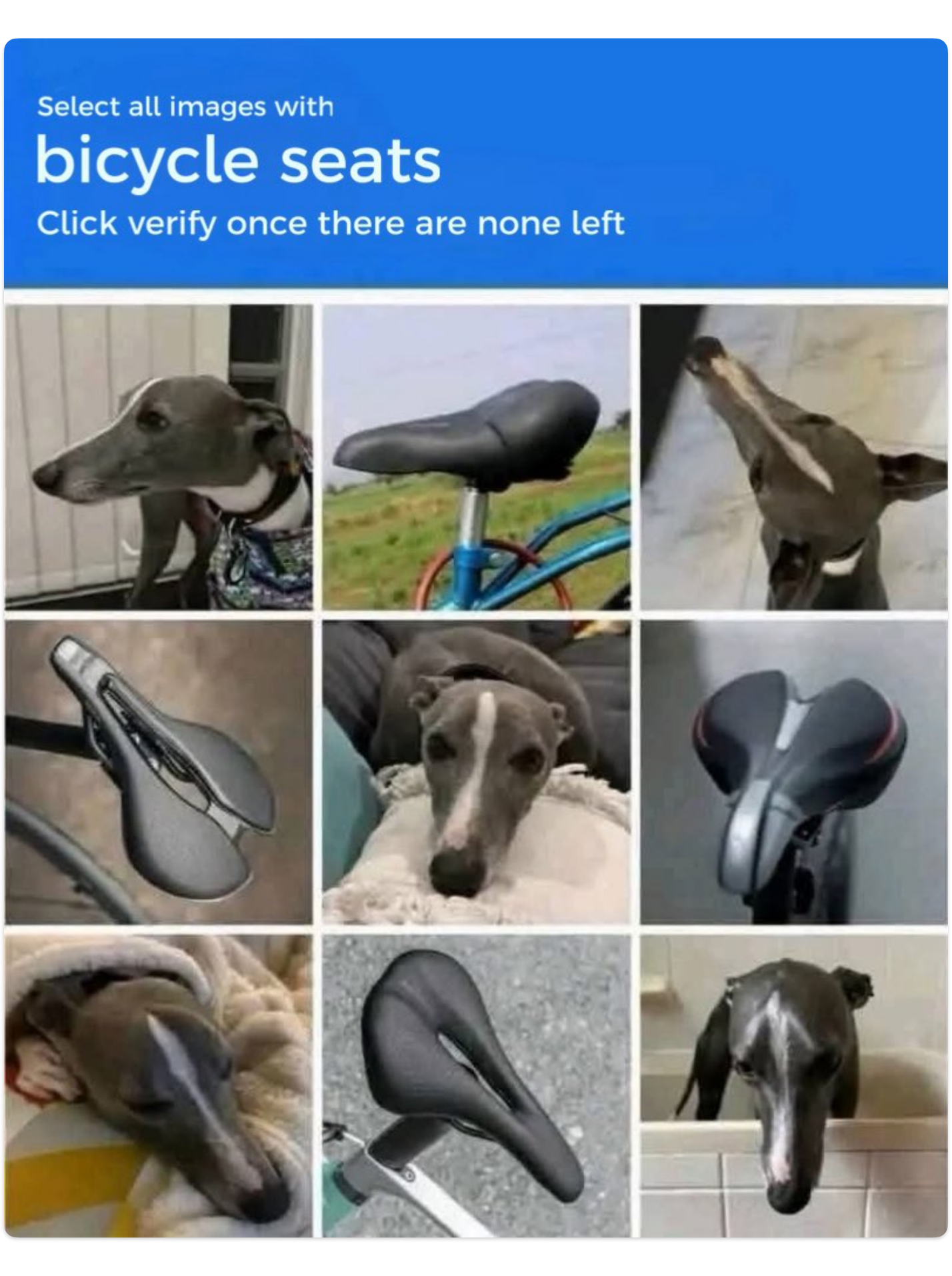
How they stack:

- Matplotlib → draws the picture
- Scikit-learn → does the learning
- Pandas → holds the table
- NumPy → does the fast math

Extra ones you will see:

- **Seaborn** — Prettier charts + sample datasets
- **imbalanced-learn** — SMOTE and other resampling techniques
- **Jupyter** — Notebooks – write code, see results, repeat

These libraries work together seamlessly. Learn one layer at a time and you can build anything!



Select all images with
bicycle seats
Click verify once there are none left

MLOps

MLOps (Machine Learning Operations) is the practice of combining Machine Learning (ML) with DevOps principles to automate, monitor, and scale the deployment of ML models in production.

Simply put MLOps is just DevOps for ML

MLOps LifeCycle

1. **Data Collection & Versioning**
2. **Model Training & Experimentation**
3. **Model Validation**
4. **Model Deployment**
5. **Monitoring & Maintenance**
6. **Continuous Integration / Continuous Deployment (CI/CD) for ML**

MLOps Tools used at various stage / lifecycle

🧰 Common Tools

Stage	Tools
Data Versioning	DVC, LakeFS
Experiment Tracking	MLflow, Weights & Biases
Model Serving	FastAPI, BentoML, Seldon, TorchServe
Pipelines	Kubeflow, Airflow, Metaflow
Deployment	Docker, Kubernetes, AWS SageMaker
Monitoring	Prometheus, Grafana, Evidently

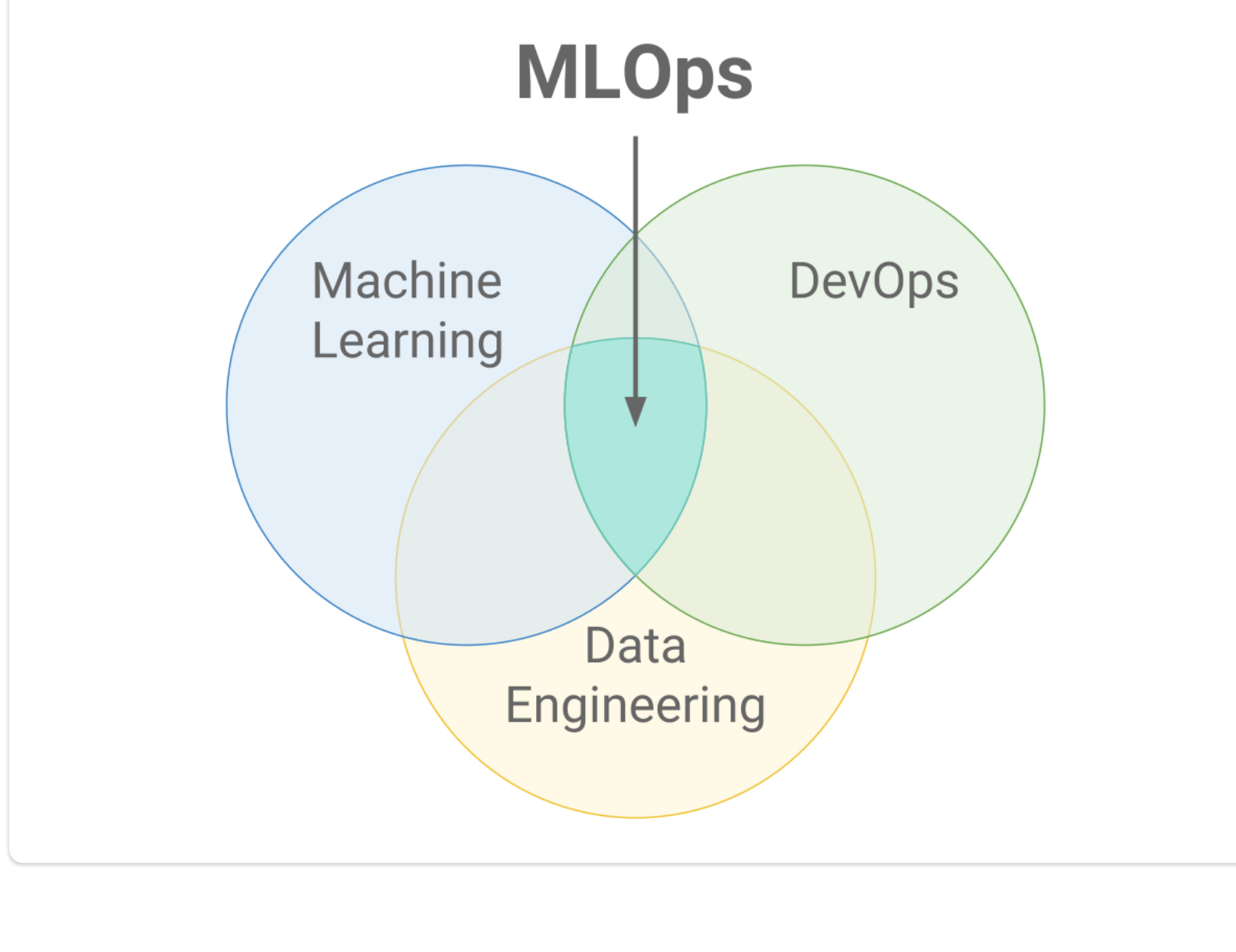
Goals of MLOps

- **Automation of training, testing, and deployment**
- **Reproducibility of experiments**
- **Scalability to handle big data/models**
- **Monitoring for model drift and performance**
- **Collaboration between data scientists, ML engineers, and ops**

What type of model are we going to build?

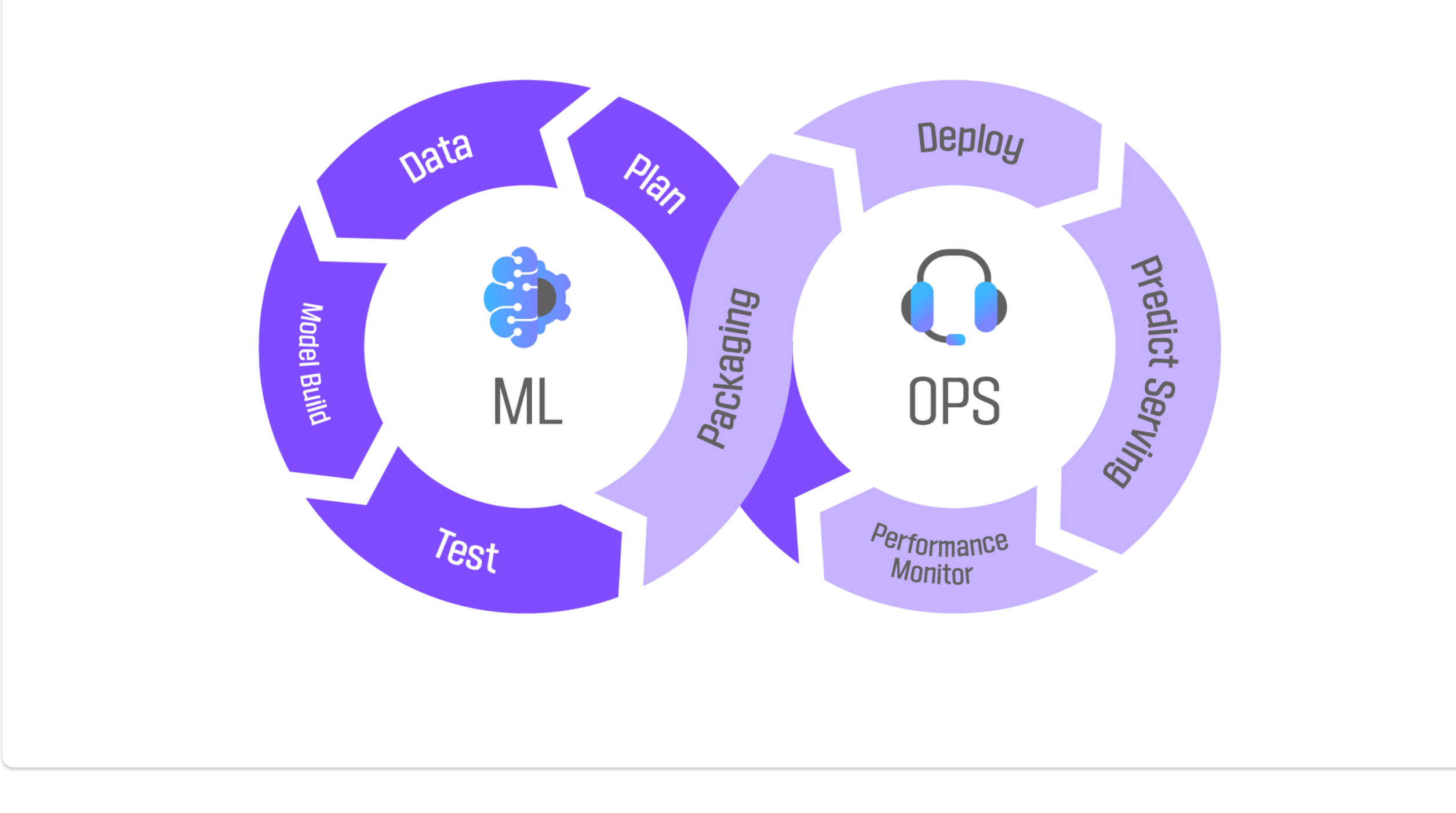
1. Prediction
2. Recommendation
3. Classification
4. Clustering

Where does MLOps fit in?



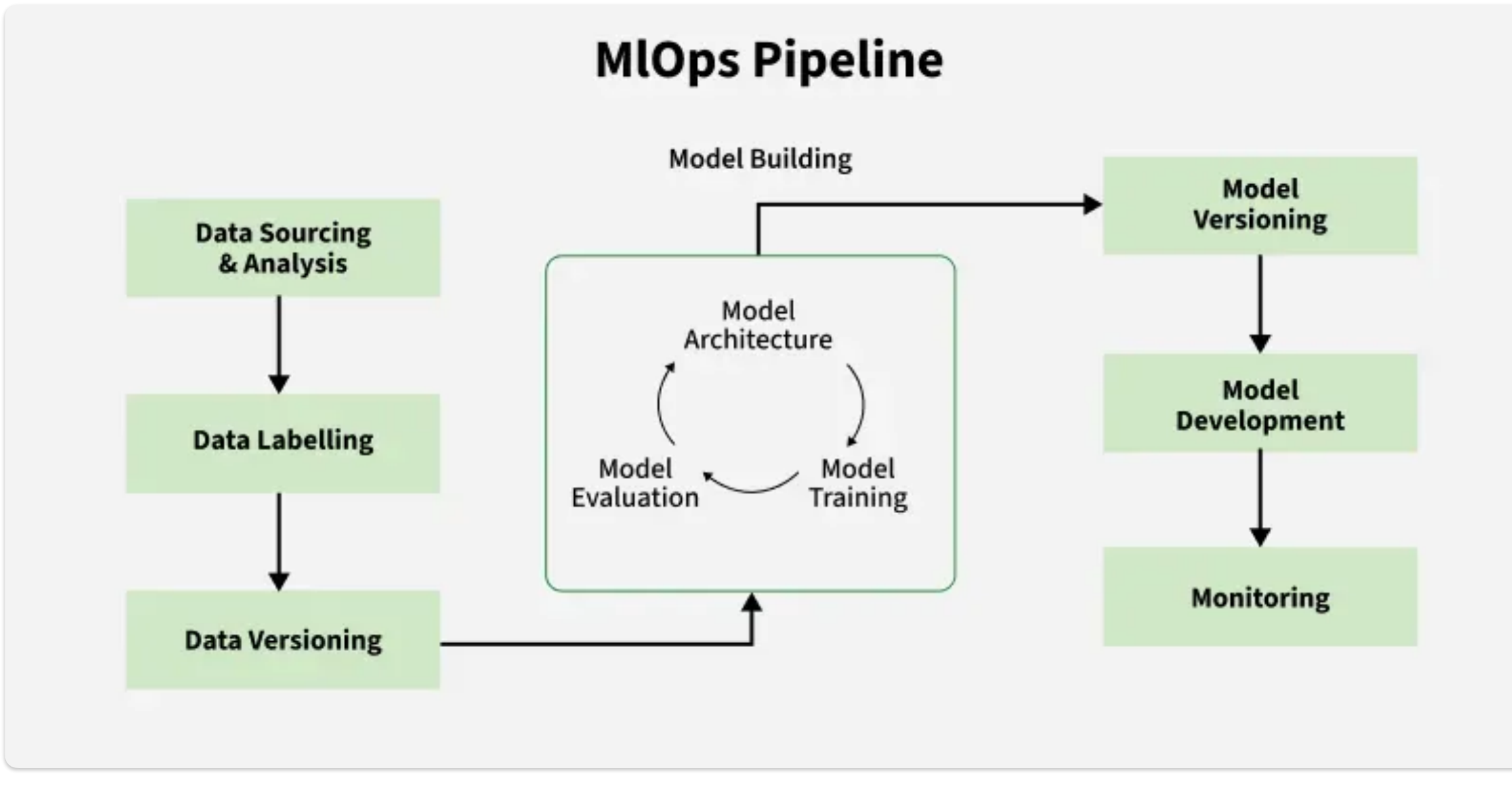
MLOps
Machine Learning
DevOps
Data Engineering

What is it?

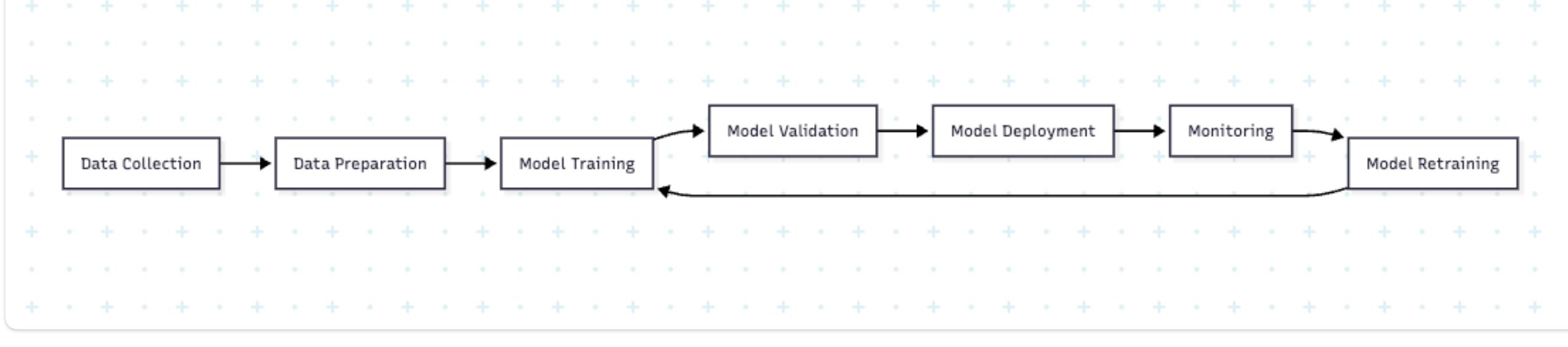


Data
Plan
Model Build
Test
ML
Packaging
Deploy
Predict Serving
Performance Monitor
OPS

The MLOps Pipeline



MlOps Pipeline
Data Sourcing & Analysis
Data Labelling
Data Versioning
Model Building
Model Architecture
Model Evaluation
Model Training
Model Versioning
Model Development
Monitoring



Data Collection
Data Preparation
Model Training
Model Validation
Model Deployment
Monitoring
Model Retraining

AI Agents at Work

sixteen_minus_nine_
16 - 9

Zuckerberg employees inventing React.js

886 likes and 26 replies

x.com